

Student name: Gordon Tse

Student ID: s223532749

SIT225: Data Capture Technologies

Activity 5.1: Firebase Realtime database

The Firebase Realtime Database is a cloud-hosted NoSQL database that lets you store and sync data between your users in real-time. Data is stored as JSON and synchronized in real-time to every connected client. In this activity, you will set up and perform operations such as queries and updates on the database using Python programming language.

Hardware Required

No hardware is required.

Software Required

Firebase Realtime database
Python 3

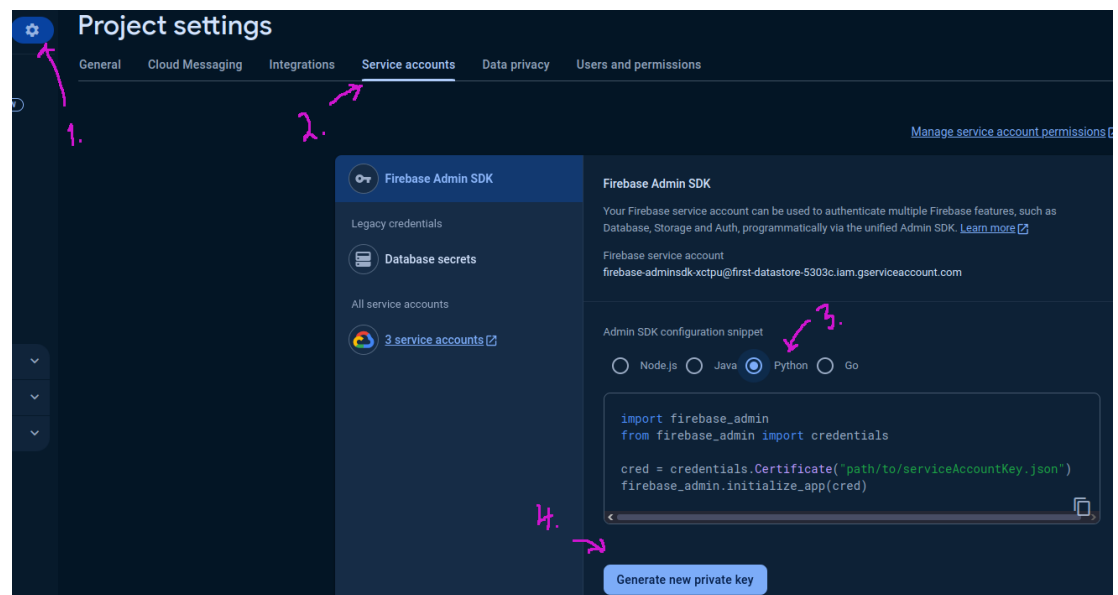
Steps

Step	Action
1	Create an Account: First, you will need to create an account in the Firebase console, follow instructions in the official Firebase document (https://firebase.google.com/docs/database/rest/start).
2	Create a Database: Follow the above Firebase document to create a database. When you click on Create Database, you have to specify the location of the database and the security rules. Two rules are available – locked mode and test mode; since we will be using the database for reading, writing, and editing, we choose test mode.
3	Setup Python library for Firebase access: We will be using Admin Database API, which is available in <i>firebase_admin</i> library. Use the below command in the command line to install. You can

follow a Firebase tutorial here (<https://www.freecodecamp.org/news/how-to-get-started-with-firebase-using-python>).

```
$ pip install firebase_admin
```

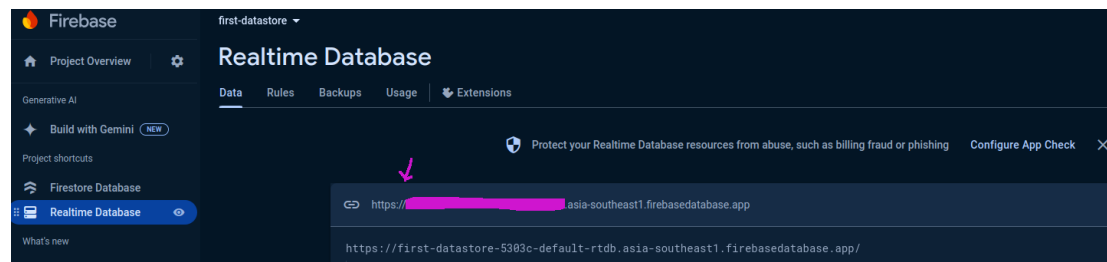
Firebase will allow access to Firebase server APIs from Google Service Accounts. To authenticate the Service Account, we require a private key in JSON format. To generate the key, go to project settings, click Generate new private key, download the file, and place it in your current folder where you will create your Python script.



- 4 **Connect to Firebase using Python version of Admin Database API:**
A credential object needs to be created to initialise the Python library which can be done using the Python code below. Python notebook can be downloaded here (https://github.com/deakin-deep-dreamer/sit225/blob/main/week_5/firebase_explore.ipynb).

```
1 import firebase_admin
2
3 databaseURL = 'https://XXX.firebaseiodatabase.app/'
4 cred_obj = firebase_admin.credentials.Certificate(
5 | 'first-datastore-5303c-firebase-adminsdk-xctpu-c9902044ac.json'
6 | )
7 default_app = firebase_admin.initialize_app(cred_obj, {
8 | 'databaseURL': databaseURL
9 | })
```

The databaseURL is a web address to reach your Firebase database that you have created in step 2. This URL can be found in the Data tab of Realtime Database.



If you compile the code snippet above, it should do with no error.

5

Write to database Using the set() Function:

We set the reference to the root of the database (or we could also set it to a key value or child key value). Data needs to be in JSON format as below.

```

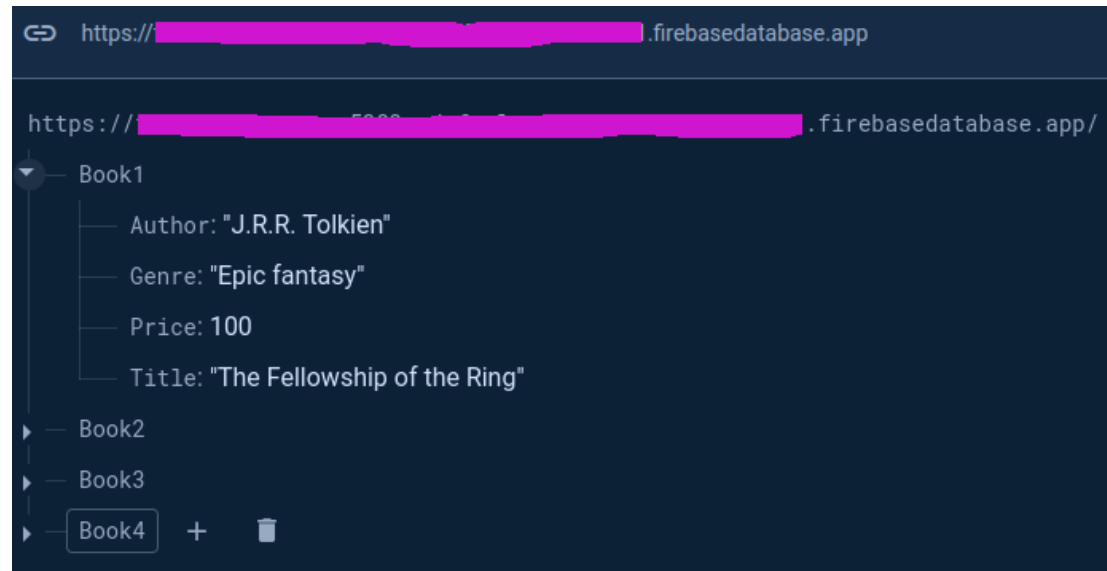
1  from firebase_admin import db
2
3  # A reference point is always needed to be set
4  # before any operation is carried out on a database.
5  #
6  ref = db.reference("/")
7
8  # JSON format data (key/value pair)
9  data = { # Outer {} contains inner data structure
10     "Book1":
11         {
12             "Title": "The Fellowship of the Ring",
13             "Author": "J.R.R. Tolkien",
14             "Genre": "Epic fantasy",
15             "Price": 100
16         },
17     "Book2":
18         {
19             "Title": "The Two Towers",
20             "Author": "J.R.R. Tolkien",
21             "Genre": "Epic fantasy",
22             "Price": 100
23         },
24     "Book3":
25         {
26             "Title": "The Return of the King",
27             "Author": "J.R.R. Tolkien",
28             "Genre": "Epic fantasy",
29             "Price": 100
30         },
31     "Book4":
32         {
33             "Title": "Brida",
34             "Author": "Paulo Coelho",
35             "Genre": "Fiction",
36             "Price": 100
37         }
38 }
39
40 # JSON format data is set (overwritten) to the reference
41 # point set at /, which is the root node.
42 #
43 ref.set(data)

```

A reference point always needed to be set where the data read/write will take place. In the code above, the reference point is set at the root of the NoSQL Document, where consider the database is a JSON tree and / is the root node

of the tree). The set() function writes (overwrites) data at the set reference point.

You can visualise the data in the Firebase console as below -



6 Read data using get() function:

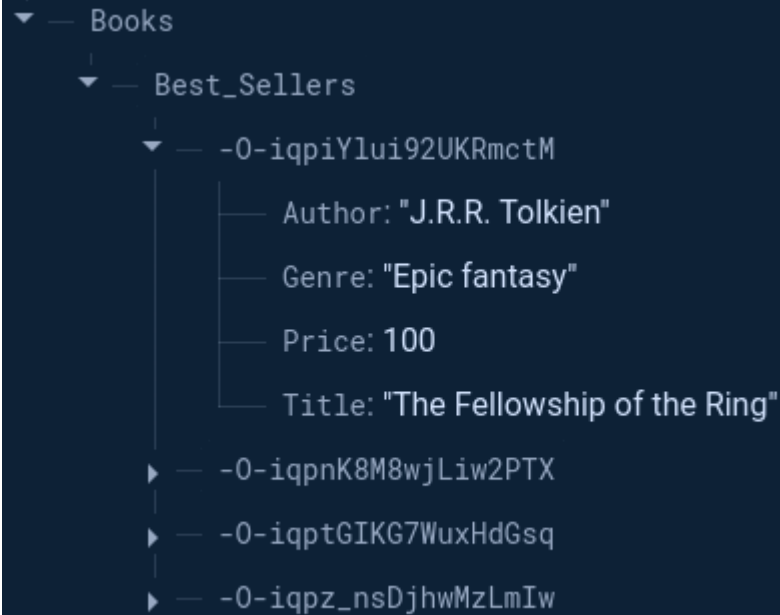
Data can be read using get() function on the reference set beforehand, as shown below.

```
1  ref = db.reference("/") # set ref point
2
3  # query all data under the ref
4  books = ref.get()
5  print(books)
6  print(type(books))
7
8  # print each item separately
9  for key, value in books.items():
10 |     print(f"{key}: {value}")
11
12
13 # Query /Book1
14 ref = db.reference("/Book1")
15 books = ref.get()
16 print(books)
```

✓ 0.3s

```
{'Book1': {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Fellowship of the Ring'},
'Book2': {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Hobbit'},
'Book3': {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Lord of the Rings'},
'Book4': {'Author': 'Paulo Coelho', 'Genre': 'Fiction', 'Price': 100, 'Title': 'Brida'}}
```

	Consider the reference set in line 1 and the output compared to the reference set at line 14 and the bottom output line to understand the use of <code>db.reference()</code> and <code>ref.get()</code> .
7	Write to database Using the push() Function: The <code>push()</code> function saves data under a <i>unique system generated key</i>. This is different than <code>set()</code> where you set the keys such as Book1, Book2, Book3 and Book4 under which the content (author, genre, price and title) appears. Let's try to push the same data in the root reference. Note that since we already has data under root / symbol, setting (or pushing) in the same reference point will eventually rewrite the original data. <pre>1 # Write using push() function 2 # Note that a set() is called on top of push() 3 # 4 ref = db.reference("/") 5 ref.set({ 6 "Books": 7 { 8 "Best_Sellers": -1 9 } 10 }) 11 12 ref = db.reference("/Books/Best_Sellers") 13 14 for key, value in data.items(): 15 ref.push().set(value)</pre> ✓ 2.0s The output will reset the previous data set in / node. The current data is shown below.



As you can see, under /Books/Best_Sellers there are 4 nodes where the node head (or node ID) is a randomly generated key which is due to the use of push() function. When data key does not matter, the use of push() function is desirable.

8

Update data:

Let's say the price of the books by J. R. R. Tolkien is reduced to 80 units to offer a discount. The first 3 books are written by this author, and we want to apply for a discount on all of them.

```

1  # Update data
2  #
3  # Requirement: The price of the books by
4  # J. R. R. Tolkien is reduced to 80 units to
5  # offer a discount.
6  #
7  ref = db.reference("/Books/Best_Sellers/")
8  best_sellers = ref.get()
9  print(best_sellers)
10 for key, value in best_sellers.items():
11     if(value["Author"] == "J.R.R. Tolkien"):
12         value["Price"] = 90
13         ref.child(key).update({"Price":80})
✓ 0.9s
  
```

As you can see, the author name is compared and the new price is set in the best_sellers dictionary and finally, an update() function is called on the ref, however, the current ref is a '/Books/Best_Sellers/', so we need to locate the

child under the ref node, so `ref.child(key)` is used in line 13. The output is shown below with a discounted price.



9

Delete data:

Let's delete all bestseller books with J.R.R. Tolkien as the author. You can locate the node using `db.reference()` (line 4) and then locate specific record (for loop in line 6) and calling `set()` with empty data `{}` as a parameter, such as `set({})`. The particular child under the ref needs to be located first by using `ref.child(key)`, otherwise, the ref node will be removed – **BE CAREFUL**.

```

1 # Let's delete all best seller books
2 # with J.R.R. Tolkien as the author.
3 #
4 ref = db.reference("/Books/Best_Sellers")
5
6 for key, value in best_sellers.items():
7     if(value["Author"] == "J.R.R. Tolkien"):
8         ref.child(key).set({})

```

This keeps only the other author data, as shown below.



If ref.child() not used, as shown the code below, all data will be removed.

```

1 ref = db.reference("/Books/Best_Sellers")
2 ref.set({})

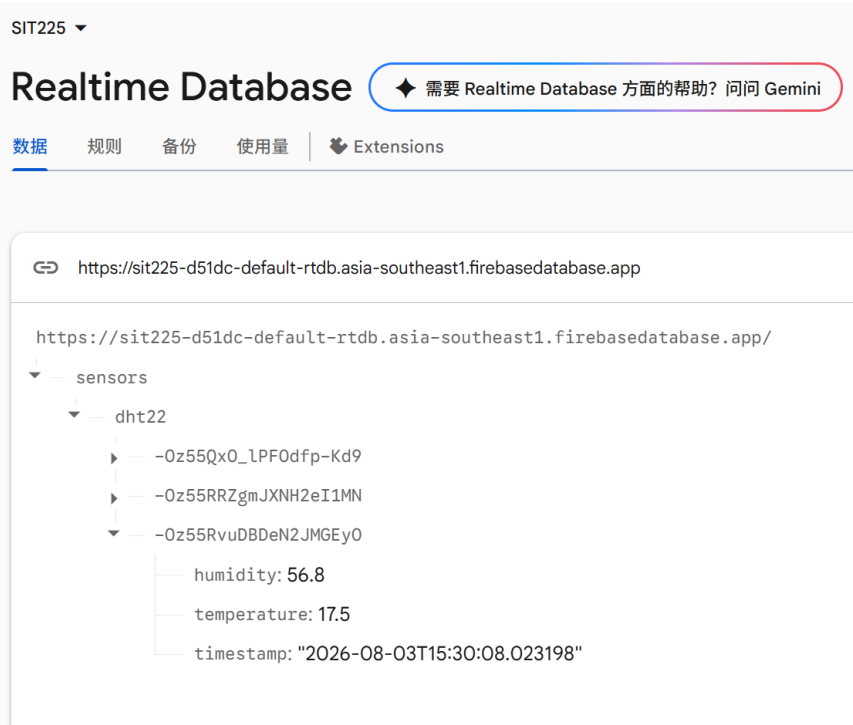
```

Now in Firebase console you will see no data exists.

10 **Question:** Run all the cells in the Notebook you have downloaded in Step 4, fill in the student information at the top cell of the Notebook. Convert the Notebook to PDF and merge with this activity sheet PDF.

Answer: Convert the Notebook to PDF and merge with this activity sheet PDF.

10 **Question:** Create a sensor data structure for DHT22 sensor which contains attributes such as sensor_name, timestamp, temperature and humidity. Remember there will be other sensors with different sensor variables such as DHT22 has 2 variables, accelerometer sensor has 3. For each such sensor, you will need to gather data over time. Discuss how you are going to handle multiple data values in JSON format? Justify your design.

	Answer: I will reuse the template code above to improve to get sensor data. For my data structure first I crate a folder called sensors to save all sensor then dht22 folder to save all record. Inside the folder it have all record using the structure “timestamp”, “temperature” and “humidity”. For adding more sensor I can just add folder in sensors.
11	Question: Generate some random data for DHT22 sensor, insert data to database, query all data and screenshot the output here. Answer:  The screenshot shows the Google Cloud Realtime Database interface for a project named SIT225. The main heading is 'Realtime Database' with a link to get help. Below the heading are tabs for '数据' (Data), '规则' (Rules), '备份' (Backups), '使用量' (Usage), and 'Extensions'. The URL bar shows the database's endpoint: https://sit225-d51dc-default-rtdb.asia-southeast1.firebaseio.com. The database structure is displayed as a tree view: 'sensors' is the root, containing a folder 'dht22'. Inside 'dht22', there are three data nodes: '-Oz55Qx0_LPF0dfp-Kd9', '-Oz55RRZgmJXNH2eI1MN', and '-Oz55RvuDBDeN2JMGEy0'. The third node is expanded, showing its data: 'humidity: 56.8', 'temperature: 17.5', and 'timestamp: "2026-08-03T15:30:08.023198"'.
12	Question: Generate some random data for the SR04 Ultrasonic sensor, insert data to database, query all data and screenshot the output here. Answer:

	Realtime Database ◆ 需要 Realtime Database 方面的帮助? 问问 Gemini 数据 规则 备份 使用量 Extensions https://sit225-d51dc-default-rtdb.asia-southeast1.firebaseio.com https://sit225-d51dc-default-rtdb.asia-southeast1.firebaseio.com/ sensors dht22 sr04 -0z5C2vJgP50Kp1X2x-y -0z5C2xJMIAT03C1rGwa -0z5C3-yFnj3bjfaGnIP distance_cm: 88.94 timestamp: "2026-08-03T15:59:00.983699"
13	Question: Firebase Realtime database generates events on data operations. You can refer to section 'Handling Realtime Database events' in the document (https://firebase.google.com/docs/functions/database-events?gen=2nd). Discuss in the active learning session and summarise the idea of database events and how it is handled using Python SDK. Note that these events are useful when your sensors (from Arduino script) store data directly to Firebase Realtime database and you would like to track data update actions from a central Python application such as a monitoring dashboard. Answer: Firebase Realtime Database is a reactive, real-time cloud database. When data operations occur such as write, update or delete, Firebase automatically triggers Database Events. What I did above is push reading like DDHT22 and SR04 sensor directly to firebase. It allows external app or device to react immediately without refreshing the database. The <code>firebase_admin</code> Python SDK handles events using Event Listeners via the <code>db.reference().listen(callback)</code> method.

Activity 5.2: Data wrangling

Data wrangling is the process of converting raw data into a usable form. The process includes collecting, processing, analyzing, and tidying the raw data so that it can be easily read and analyzed. In this activity, you will use the common library in python, "pandas".

Hardware Required

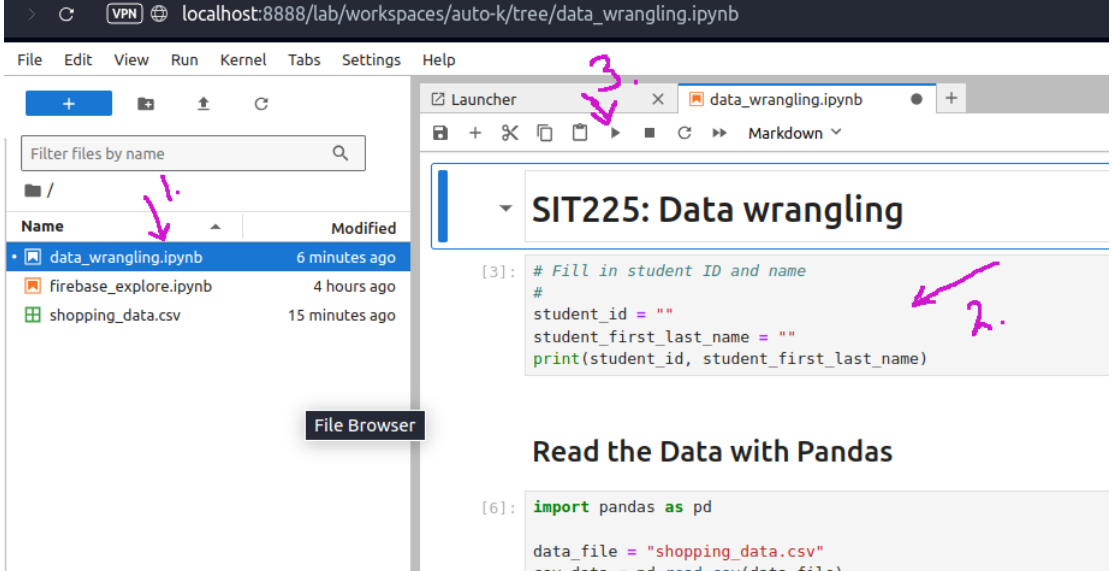
No hardware is required.

Software Required

Python 3
Pandas Python library

Steps

Step	Action
1	Install Pandas using the command below. Most likely you already have Pandas installed if you have installed Python using Anaconda distribution (https://www.anaconda.com/download). <pre>\$ pip install pandas</pre> A Python notebook is shared in the GitHub link (https://github.com/deakin-deep-dreamer/sit225/tree/main/week_5). There will be a data_wrangling.ipynb, shopping_data.csv and shopping_data_missingvalue.csv files among others. Download the week_5 folder in your computer, open a command prompt in that folder, and write the command below in the command line: <pre>\$ jupyter lab</pre> This will open Python Jupyter Notebook where in the left panel you can see the files (labeled as 1 in figure).

	 Each cell contains Python code (labeled as 2 in figure), you can run a cell by clicking on the cell, so the cursor appears in that cell and then click on the play button at the top of the panel (labeled as 3 in the figure).
2	Question: Run each cell to produce output. Follow instructions in the notebook to complete codes in some of the cells. Convert the notebook to PDF from menu File > Save and Export Notebook As > PDF. Convert this activity sheet to PDF and merge with the notebook PDF. Answer: There is no answer to write here. You have to answer in the Jupyter Notebook.
3	Question: Once you went through the cells in the Notebook, you now have a basic understanding of data wrangling. Pandas are a powerful tool and can be used for reading CSV data. Can you use Pandas in reading sensor CSV data that you generated earlier? Describe if any modification you think necessary? Answer: I think it possible to read the data we just input in firebase however I need to decide a way to get the data first like should I export it to csv file first or I can directly get from the database? After getting the data I also need to check if any Nan value and clean them. further I can change my timeframe format.
4	Question: What do you understand of the Notebook section called Handling Missing Value? Discuss in group and briefly summarise different missing value imputation methods and their applicability on different data conditions.

	Answer: if a dataset have missing value or NAN value, some function like <code>mean()</code> or math calculation won't work and return error. So we can either delete the or using so math method to fill in the gap without changing too much of overall performance. Like using <code>max()</code>, <code>min()</code>, <code>mean</code>, <code>median()</code> or <code>avg()</code>. Depend on which is most suitable for the project.
--	---

```
In [23]: # Fill in student ID and name  
#  
student_id = "s223532749"  
student_first_last_name = "Gordon Tse"  
print(student_id, student_first_last_name)
```

s223532749 Gordon Tse

```
In [ ]: # Install libraris, if not yet.
# ! pip install firebase_admin pandas

import firebase_admin

databaseURL = 'https://sit225-d51dc-default-rtdb.asia-
southeast1.firebaseio.com/'
cred_obj = firebase_admin.credentials.Certificate(
    'sit225-d51dc-firebase-adminsdk-fbsvc-bb6c4788ce.json'
)
default_app = firebase_admin.initialize_app(cred_obj, {
    'databaseURL':databaseURL
})
```

```
-----
-----
ValueError                                Traceback (most
recent call last)
Cell In[24], line 10
      6 databaseURL = 'https://sit225-d51dc-default-rtdb.asia-
southeast1.firebaseio.com/'
      7 cred_obj = firebase_admin.credentials.Certificate(
      8     'sit225-d51dc-firebase-adminsdk-fbsvc-
bb6c4788ce.json'
      9 )
--> 10 default_app = firebase_admin.initialize_app(cred_obj, {
      11     'databaseURL':databaseURL
      12     })

File c:\Users\gordo\anaconda3\Lib\site-
packages\firebase_admin\__init__.py:74, in
initialize_app(credential, options, name)
      71     return app
      73 if name == _DEFAULT_APP_NAME:
--> 74     raise ValueError((
      75         'The default Firebase app already exists. This
means you called '
      76         'initialize_app() more than once without
providing an app name as '
      77         'the second argument. In most cases you only
need to call '
      78         'initialize_app() once. But if you do want to
initialize multiple '
      79         'apps, pass a second argument to
initialize_app() to give each app '
      80         'a unique name.'))
      82 raise ValueError(
      83     f'Firebase app named "{name}" already exists. This
means you called '
      84     'initialize_app() more than once with the same app
name as the '
      85     'second argument. Make sure you provide a unique
name every time '

```

86 'you call initialize_app().')

ValueError: The default Firebase app already exists. This means you called initialize_app() more than once without providing an app name as the second argument. In most cases you only need to call initialize_app() once. But if you do want to initialize multiple apps, pass a second argument to initialize_app() to give each app a unique name.

```
In [25]: from firebase_admin import db

# A reference point is always needed to be set
# before any operation is carried out on a database.
#
ref = db.reference("/")

# JSON format data (key/value pair)
data = { # Outer {} contains inner data structure
    "Book1":
        {
            "Title": "The Fellowship of the Ring",
            "Author": "J.R.R. Tolkien",
            "Genre": "Epic fantasy",
            "Price": 100
        },
    "Book2":
        {
            "Title": "The Two Towers",
            "Author": "J.R.R. Tolkien",
            "Genre": "Epic fantasy",
            "Price": 100
        },
    "Book3":
        {
            "Title": "The Return of the King",
            "Author": "J.R.R. Tolkien",
            "Genre": "Epic fantasy",
            "Price": 100
        },
    "Book4":
        {
            "Title": "Brida",
            "Author": "Paulo Coelho",
            "Genre": "Fiction",
            "Price": 100
        }
}

# JSON format data is set (overwritten) to the reference
# point set at /, which is the root node.
#
ref.set(data)
```

In [26]: `ref = db.reference("/") # set ref point`

```
# query all data under the ref
books = ref.get()
print(books)
print(type(books))

# print each item separately
for key, value in books.items():
    print(f"{key}: {value}")
```

```
# Query /Book1
ref = db.reference("/Book1")
books = ref.get()
print(books)
```

```
{'Book1': {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy',
'Price': 100, 'Title': 'The Fellowship of the Ring'}, 'Book2':
{'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price':
100, 'Title': 'The Two Towers'}, 'Book3': {'Author': 'J.R.R.
Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The
Return of the King'}, 'Book4': {'Author': 'Paulo Coelho', 'Genre':
'Fiction', 'Price': 100, 'Title': 'Brida'}}
<class 'dict'>
Book1: {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy',
'Price': 100, 'Title': 'The Fellowship of the Ring'}
Book2: {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy',
'Price': 100, 'Title': 'The Two Towers'}
Book3: {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy',
'Price': 100, 'Title': 'The Return of the King'}
Book4: {'Author': 'Paulo Coelho', 'Genre': 'Fiction', 'Price':
100, 'Title': 'Brida'}
{'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price':
100, 'Title': 'The Fellowship of the Ring'}
```

In [27]: `# Write using push() function`

```
# Note that a set() is called on top of push()
#
ref = db.reference("/")
ref.set({
    "Books":
        {
            "Best_Sellers": -1
        }
})

ref = db.reference("/Books/Best_Sellers")

for key, value in data.items():
    ref.push().set(value)
```

```
In [28]: # Update data
#
# Requirement: The price of the books by
# J. R. R. Tolkien is reduced to 80 units to
# offer a discount.
#
ref = db.reference("/Books/Best_Sellers/")
best_sellers = ref.get()
print(best_sellers)
for key, value in best_sellers.items():
    if(value["Author"] == "J.R.R. Tolkien"):
        value["Price"] = 90
        ref.child(key).update({"Price":80})

{'-Oyquin0tTFaJI1rm1GK': {'Author': 'J.R.R. Tolkien', 'Genre':
'Epic fantasy', 'Price': 100, 'Title': 'The Fellowship of the
Ring'}, '-OyquirFUS24HgPGUAjn': {'Author': 'J.R.R. Tolkien',
'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Two Towers'},
'-OyquivKF_9LBVnsEz7r': {'Author': 'J.R.R. Tolkien', 'Genre':
'Epic fantasy', 'Price': 100, 'Title': 'The Return of the King'},
'-OyquizJown7vK-kl0zy': {'Author': 'Paulo Coelho', 'Genre':
'Fiction', 'Price': 100, 'Title': 'Brida'}}
```

```
In [29]: # Let's delete all best seller books
# with J.R.R. Tolkien as the author.
#
ref = db.reference("/Books/Best_Sellers")

for key, value in best_sellers.items():
    if(value["Author"] == "J.R.R. Tolkien"):
        ref.child(key).set({})
```

```
In [30]: # Delete all best_seller data.
#
ref = db.reference("/Books/Best_Sellers/")
best_sellers = ref.get()
print(best_sellers)
print(type(best_sellers))

{'-OyquizJown7vK-kl0zy': {'Author': 'Paulo Coelho', 'Genre':
'Fiction', 'Price': 100, 'Title': 'Brida'}}
<class 'dict'>
```

```
In [31]: ref = db.reference("/Books/Best_Sellers")
ref.set({})
```

SIT225: Data wrangling

Run each cell to generate output and finally convert this notebook to PDF.

```
# Fill in student ID and name
#
student_id = "Gordon"
student_first_last_name = "Tse"
print(student_id, student_first_last_name)
```

Gordon Tse

Read the Data with Pandas

Pandas has a dedicated function `read_csv()` to read CSV files.

Just in case we have a large number of data, we can just show into only five rows with `head` function. It will show you 5 rows data automatically.

```
import pandas as pd

data_file = "shopping_data.csv"
csv_data = pd.read_csv(data_file)

print(csv_data)

# show into only five rows with head function
print(csv_data.head())
```

	CustomerID	Genre	Age	Annual Income (k\$)	Spending Score (1-100)
0	1	Male	19	15	
1	2	Male	21	15	
2	3	Female	20	16	
3	4	Female	23	16	
4	5	Female	31	17	
...	...	...	...	...	...
195	196	Female	35	120	
196	197	Female	45	126	

197	198	Male	32	126
74				
198	199	Male	32	137
18				
199	200	Male	30	137
83				

```
[200 rows x 5 columns]
```

	CustomerID	Genre	Age	Annual Income (k\$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

Access the Column

Pandas has provided function `.columns` to access the column of the data source.

```
print(csv_data.columns)

# if we want to access just one column, for example "Age"
print("Age:")
print(csv_data["Age"])

Index(['CustomerID', 'Genre', 'Age', 'Annual Income (k$)',
      'Spending Score (1-100)'],
      dtype='object')
Age:
0      19
1      21
2      20
3      23
4      31
...
195    35
196    45
197    32
198    32
199    30
Name: Age, Length: 200, dtype: int64
```

Access the Row

In addition to accessing data through columns, using pandas can also access using rows. In contrast to access through columns, the function to display data from a row is the `.iloc[i]` function where `[i]` indicates the order of the rows to be displayed where the index starts from 0.

```
# we want to know what line 5 contains
print(csv_data.iloc[5])

print()

# We can combine both of those function to show row and column we
# want.
# For the example, we want to show the value in column "Age" at the
# first row
# (remember that the row starts at 0)
#
print(csv_data["Age"].iloc[1])
```

CustomerID	6
Genre	Female
Age	22
Annual Income (k\$)	17
Spending Score (1-100)	76

Name: 5, dtype: object

21

Show Data Based on Range

After displaying a data set, what if you want to display data from rows 5 to 20 of a dataset? To anticipate this, pandas can also display data within a certain range, both ranges for rows only, only columns, and ranges for rows and columns

```
print("Shows data to 5th to less than 10th in a row:")
print(csv_data.iloc[5:10])
```

Shows data to 5th to less than 10th in a row:

	CustomerID	Genre	Age	Annual Income (k\$)	Spending Score (1-100)
5	6	Female	22	17	76
6	7	Female	35	18	6
7	8	Female	23	18	94
8	9	Male	64	19	3
9	10	Female	30	19	72

Using Numpy to Show the Statistic Information

The describe() function allows to quickly find statistical information from a dataset. Those information such as mean, median, modus, max min, even standard deviation. Don't forget to install Numpy before using describe function.

```
print(csv_data.describe(include="all"))
```

	CustomerID	Genre	Age	Annual Income (k\$)	\
count	200.000000	200	200.000000	200.000000	
unique	NaN	2	NaN	NaN	
top	NaN	Female	NaN	NaN	
freq	NaN	112	NaN	NaN	
mean	100.500000	NaN	38.850000	60.560000	
std	57.879185	NaN	13.969007	26.264721	
min	1.000000	NaN	18.000000	15.000000	
25%	50.750000	NaN	28.750000	41.500000	
50%	100.500000	NaN	36.000000	61.500000	
75%	150.250000	NaN	49.000000	78.000000	
max	200.000000	NaN	70.000000	137.000000	

	Spending Score (1-100)
count	200.000000
unique	NaN
top	NaN
freq	NaN
mean	50.200000
std	25.823522
min	1.000000
25%	34.750000
50%	50.000000
75%	73.000000
max	99.000000

Handling Missing Value

For the first step, we will figure out if there is missing value.

```
print(csv_data.isnull().values.any())
```

```
print()
```

False

We will use another data source with missing values to practice this part.

```
data_missing = pd.read_csv("shopping_data_missingvalue.csv")
```

```
print(data_missing.head())
```

```
print()
```

```
print("Missing? ", data_missing.isnull().values.any())
```

	CustomerID	Genre	Age	Annual Income (k\$)	Spending Score (1-100)
0	1	Male	19.0	15.0	39.0
1	2	Male	NaN	15.0	

81.0				
2	3	Female	20.0	NaN
6.0				
3	4	Female	23.0	16.0
77.0				
4	5	Female	31.0	17.0
NaN				

Missing? True

Ways to deal with missing values.

Follow the tutorial (<https://deepnote.com/app/rickyharyanto14-3390/Data-Wrangling-w-Python-e5d1a23e-33cf-416d-ad27-4c3f7f467442>). It includes -

1. Delete data
 - deleting rows
 - pairwise deletion
 - delete column
2. imputation
 - time series problem
 - Data without trend with seasonality (mean, median, mode, random)
 - Data with trend and without seasonality (linear interpolation)
 - general problem
 - Data categorical (Make NA as multiple imputation)
 - Data numerical or continuous (mean, median, mode, multiple imputation and linear regression)

Filling with Mean Values

The mean is used for data that has a few outliers/noise/anomalies in the distribution of the data and its contents. This value will later fill in the empty value of the dataset that has a missing value case. To fill in an empty value use the `fillna()` function

```
print(data_missing.mean())
```

```
"""
```

Question: This code will generate error. Can you explain why and how it can be solved?

Move on to the next cell to find one way it can be solved.

Answer: The code will generate an error because `mean()` function is being called on a Dataset that contains missing values. The `mean()` function cannot compute the mean of columns that have NaN values, which leads to an error.

```
"""
```

```
-----  
-----  
TypeError                                Traceback (most recent call  
last)
```

```
Cell In[12], line 1
```

```
----> 1 print(data_missing.mean())  
      3 ""  
      4  
      5 Question: This code will generate error. Can you explain why  
and how it can be solved?  
      (...)   
      9  
     10 ""
```

```
File c:\Users\gordo\anaconda3\Lib\site-packages\pandas\core\  
frame.py:11335, in DataFrame.mean(self, axis, skipna, numeric_only,  
**kwargs)
```

```
    11327 @doc(make_doc("mean", ndim=2))  
    11328 def mean(  
    11329     self,  
    (...)   
    11333     **kwargs,  
    11334 ):  
> 11335     result = super().mean(axis, skipna, numeric_only,  
**kwargs)  
    11336     if isinstance(result, Series):  
    11337         result = result.__finalize__(self, method="mean")
```

```
File c:\Users\gordo\anaconda3\Lib\site-packages\pandas\core\  
generic.py:11992, in NDFrame.mean(self, axis, skipna, numeric_only,  
**kwargs)
```

```
    11985 def mean(  
    11986     self,  
    11987     axis: Axis | None = 0,  
    (...)   
    11990     **kwargs,  
    11991 ) -> Series | float:  
> 11992     return self._stat_function(  
    11993         "mean", nanops.nanmean, axis, skipna, numeric_only,  
**kwargs  
    11994     )
```

```
File c:\Users\gordo\anaconda3\Lib\site-packages\pandas\core\  
generic.py:11949, in NDFrame._stat_function(self, name, func, axis,  
skipna, numeric_only, **kwargs)
```

```
    11945 nv.validate_func(name, (), kwargs)  
    11947 validate_bool_kwarg(skipna, "skipna", none_allowed=False)  
> 11949 return self._reduce(  
    11950     func, name=name, axis=axis, skipna=skipna,  
numeric_only=numeric_only
```

```
11951 )
```

```
File c:\Users\gordo\anaconda3\Lib\site-packages\pandas\core\
frame.py:11204, in DataFrame._reduce(self, op, name, axis, skipna,
numeric_only, filter_type, **kwargs)
```

```
11200     df = df.T
11202 # After possibly _get_data and transposing, we are now in the
11203 # simple case where we can use BlockManager.reduce
> 11204 res = df._mgr.reduce(blk_func)
11205 out = df._constructor_from_mgr(res, axes=res.axes).iloc[0]
11206 if out_dtype is not None and out.dtype != "boolean":
```

```
File c:\Users\gordo\anaconda3\Lib\site-packages\pandas\core\internals\
managers.py:1459, in BlockManager.reduce(self, func)
```

```
1457 res_blocks: list[Block] = []
1458 for blk in self.blocks:
-> 1459     nbs = blk.reduce(func)
1460     res_blocks.extend(nbs)
1462 index = Index([None]) # placeholder
```

```
File c:\Users\gordo\anaconda3\Lib\site-packages\pandas\core\internals\
blocks.py:377, in Block.reduce(self, func)
```

```
371 @final
372 def reduce(self, func) -> list[Block]:
373     # We will apply the function and reshape the result into a
single-row
374     # Block with the same mgr_locs; squeezing will be done at
a higher level
375     assert self.ndim == 2
--> 377     result = func(self.values)
379     if self.values.ndim == 1:
380         res_values = result
```

```
File c:\Users\gordo\anaconda3\Lib\site-packages\pandas\core\
frame.py:11136, in DataFrame._reduce.<locals>.blk_func(values, axis)
```

```
11134     return np.array([result])
11135 else:
> 11136     return op(values, axis=axis, skipna=skipna, **kwargs)
```

```
File c:\Users\gordo\anaconda3\Lib\site-packages\pandas\core\
nanops.py:147, in bottleneck_switch.__call__.<locals>.f(values, axis,
skipna, **kwargs)
```

```
145     result = alt(values, axis=axis, skipna=skipna, **kwargs)
146 else:
--> 147     result = alt(values, axis=axis, skipna=skipna, **kwargs)
149 return result
```

```
File c:\Users\gordo\anaconda3\Lib\site-packages\pandas\core\
nanops.py:404, in _datetimelike_compat.<locals>.new_func(values, axis,
skipna, mask, **kwargs)
```

```

    401 if datetimelike and mask is None:
    402     mask = isna(values)
--> 404 result = func(values, axis=axis, skipna=skipna, mask=mask,
**kwargs)
    406 if datetimelike:
    407     result = _wrap_results(result, orig_values.dtype,
fill_value=iNaT)

```

```

File c:\Users\gordo\anaconda3\Lib\site-packages\pandas\core\
nanops.py:720, in nanmean(values, axis, skipna, mask)
    718 count = _get_counts(values.shape, mask, axis,
dtype=dtype_count)
    719 the_sum = values.sum(axis, dtype=dtype_sum)
--> 720 the_sum = _ensure_numeric(the_sum)
    722 if axis is not None and getattr(the_sum, "ndim", False):
    723     count = cast(np.ndarray, count)

```

```

File c:\Users\gordo\anaconda3\Lib\site-packages\pandas\core\
nanops.py:1678, in _ensure_numeric(x)
    1675 inferred = lib.infer_dtype(x)
    1676 if inferred in ["string", "mixed"]:
    1677     # GH#44008, GH#36703 avoid casting e.g. strings to numeric
-> 1678     raise TypeError(f"Could not convert {x} to numeric")
    1679 try:
    1680     x = x.astype(np.complex128)

```

TypeError: Could not convert

```

['MaleMaleFemaleFemaleFemaleFemaleFemaleFemaleMaleFemaleMaleFemaleFema
leFemaleMaleMaleFemaleMaleMaleFemaleMaleMaleFemaleMaleFemaleMaleFemale
MaleFemaleFemaleMaleFemaleMaleMaleFemaleFemaleFemaleFemaleFemaleFemale
FemaleMaleMaleFemaleFemaleFemaleFemaleFemaleFemaleFemaleFemaleFemaleMaleFema
leMaleFemaleMaleFemaleMaleFemaleMaleMaleMaleFemaleFemaleMaleMaleFemale
FemaleMaleFemaleMaleFemaleFemaleFemaleMaleMaleFemaleMaleFemaleFemaleMa
leMaleMaleFemaleFemaleMaleFemaleFemaleFemaleFemaleFemaleFemaleMaleMaleFemale
FemaleMaleFemaleFemaleMaleMaleFemaleFemaleMaleMaleMaleFemaleFemaleMale
MaleMaleMaleFemaleFemaleMaleFemaleFemaleFemaleFemaleFemaleFemaleMaleFe
maleFemaleMaleFemaleFemaleMaleMaleMaleMaleMaleFemaleFemaleMaleFema
leFemaleMaleMaleFemaleFemaleMaleFemaleFemaleMaleMaleMaleFemaleFemaleMa
leMaleMaleFemaleFemaleFemaleFemaleMaleFemaleMaleFemaleFemaleFemaleMale
FemaleMaleFemaleMaleFemaleFemaleMaleMaleMaleMaleMaleFemaleFemaleMaleMa
leMaleMaleFemaleFemaleMaleFemaleFemaleMaleFemaleMaleFemaleFemaleFemale
FemaleMaleFemaleFemaleFemaleFemaleFemaleMaleMaleMale'] to numeric

```

Genre column contains string values and numerical operation mean fails.

Lets drop Genre column since for numerical calculation.

#

```

data_missing_wo_genre = data_missing.drop(columns=['Genre'])
print(data_missing_wo_genre.head())

```

	CustomerID	Age	Annual Income (k\$)	Spending Score (1-100)
0	1	19.0	15.0	39.0
1	2	NaN	15.0	81.0
2	3	20.0	NaN	6.0
3	4	23.0	16.0	77.0
4	5	31.0	17.0	NaN

```
print(data_missing_wo_genre.mean())
```

```
CustomerID      100.500000
Age             38.939698
Annual Income (k$) 61.005051
Spending Score (1-100) 50.489899
dtype: float64
```

```
print("Dataset with empty values! :")
```

```
print(data_missing_wo_genre.head(10))
```

```
data_filling=data_missing_wo_genre.fillna(data_missing_wo_genre.mean()
)
```

```
print("Dataset that has been processed Handling Missing Values with
Mean :")
```

```
print(data_filling.head(10))
```

```
# Observe the missing value imputation in corresponding rows.
```

```
#
```

```
Dataset with empty values! :
```

	CustomerID	Age	Annual Income (k\$)	Spending Score (1-100)
0	1	19.0	15.0	39.0
1	2	NaN	15.0	81.0
2	3	20.0	NaN	6.0
3	4	23.0	16.0	77.0
4	5	31.0	17.0	NaN
5	6	22.0	NaN	76.0
6	7	35.0	18.0	6.0
7	8	23.0	18.0	94.0
8	9	64.0	19.0	NaN
9	10	30.0	19.0	72.0

```
Dataset that has been processed Handling Missing Values with Mean :
```

	CustomerID	Age	Annual Income (k\$)	Spending Score (1-100)
0	1	19.000000	15.000000	39.000000
1	2	38.939698	15.000000	81.000000
2	3	20.000000	61.005051	6.000000
3	4	23.000000	16.000000	77.000000
4	5	31.000000	17.000000	50.489899
5	6	22.000000	61.005051	76.000000
6	7	35.000000	18.000000	6.000000
7	8	23.000000	18.000000	94.000000

8	9	64.000000	19.000000	50.489899
9	10	30.000000	19.000000	72.000000

Filling with Median

The median is used when the data presented has a high outlier. The median was chosen because it is the middle value, which means it is not the result of calculations involving outlier data. In some cases, outlier data is considered disturbing and often considered noisy because it can affect class distribution and interfere with clustering analysis.

```
print(data_missing_wo_genre.median())
print("Dataset with empty values! :")
print(data_missing_wo_genre.head(10))

data_filling2=data_missing_wo_genre.fillna(data_missing_wo_genre.median())
print("Dataset that has been processed Handling Missing Values with Median :")
print(data_filling2.head(10))
```

Observe the missing value imputation in corresponding rows.
#

```
CustomerID      100.5
Age              36.0
Annual Income (k$)  62.0
Spending Score (1-100)  50.0
dtype: float64
```

Dataset with empty values! :

	CustomerID	Age	Annual Income (k\$)	Spending Score (1-100)
0	1	19.0	15.0	39.0
1	2	NaN	15.0	81.0
2	3	20.0	NaN	6.0
3	4	23.0	16.0	77.0
4	5	31.0	17.0	NaN
5	6	22.0	NaN	76.0
6	7	35.0	18.0	6.0
7	8	23.0	18.0	94.0
8	9	64.0	19.0	NaN
9	10	30.0	19.0	72.0

Dataset that has been processed Handling Missing Values with Median :

	CustomerID	Age	Annual Income (k\$)	Spending Score (1-100)
0	1	19.0	15.0	39.0
1	2	36.0	15.0	81.0
2	3	20.0	62.0	6.0
3	4	23.0	16.0	77.0
4	5	31.0	17.0	50.0
5	6	22.0	62.0	76.0
6	7	35.0	18.0	6.0

7	8	23.0	18.0	94.0
8	9	64.0	19.0	50.0
9	10	30.0	19.0	72.0